



COS40007

Portfolio Assessment 2

Studio 1 - 3

Duc Thuan Tran|104330455

Table of Contents

Overview	2
Summary Table from Studio 3.....	3
Activity 6 Table.....	3
Activity 7 Table.....	3
Step 1: Data Collection	3
Step 2: Composite Columns.....	5
Step 3: Data Pre-processing	6
Step 4: Training	8
Step 5: Model Selection	13
Appendix: Source Code Repository	14
References.....	14

Overview

This project implements a systematic approach to developing a machine learning model for classifying meat processing activities using acceleration data from body-worn sensors. The dataset contains measurements from 17 sensors positioned on different body parts during boning and slicing tasks. By focusing on right and left upper leg sensors based on student ID requirements, the analysis extracts meaningful motion patterns that differentiate between these activities. Through structured data preprocessing, composite feature engineering, and statistical transformation, the raw acceleration time-series data was converted into 108 descriptive features capturing movement characteristics per minute of activity. Multiple classification models were evaluated, including various SVM configurations, Random Forest, SGD, and Neural Networks, with hyperparameter tuning and dimensionality reduction techniques. The results demonstrated that a properly configured SVM classifier could achieve over 90% accuracy in distinguishing between boning and slicing activities, outperforming more complex approaches. This work highlights the effectiveness of methodical

feature engineering in sensor-based activity recognition and shows how targeted sensor placement can provide sufficient information for accurate classification in industrial movement analysis.

Summary Table from Studio 3

Activity 6 Table

SVM Model Summary Table:

	Model	Train-test split	Cross-validation
0	Original features	88.82%	89.18%
1	With hyper-parameter tuning	83.69%	83.57%
2	With feature selection and hyper-parameter tuning	83.95%	83.75%
3	With PCA and hyper-parameter tuning	83.78%	83.58%

Figure 1. SVM Model Summary Table.

The SVM Model Summary Table illustrates the performance of different SVM configurations explored in Studio 3. The model using original features performed best with 88.82% accuracy for train-test split and 89.18% for cross-validation. Interestingly, hyperparameter tuning resulted in slightly lower accuracy at 83.69% and 83.57% respectively, suggesting that the default parameters were already well-suited for this dataset. Feature selection with hyperparameter tuning achieved 83.95% (train-test) and 83.75% (cross-validation), while PCA with hyperparameter tuning showed similar performance at 83.78% and 83.58%.

Activity 7 Table

Classifier Summary Table:

	Model	Train-test split	Cross-validation
0	SVM	83.69%	83.57%
1	SGD	88.25%	87.83%
2	RandomForest	92.35%	92.56%
3	MLP	89.54%	82.73%

Figure 2. Classifier Summary Table.

The Classifier Summary Table compares various machine learning algorithms applied to the same classification problem. RandomForest clearly outperformed other approaches with outstanding accuracy of 92.35% for train-test split and 92.56% for cross-validation. MLP neural network achieved the second-best train-test performance with 89.54%, though its cross-validation score was lower at 82.73%. The SGD classifier showed consistent performance with 88.25% and 87.83% accuracy, while SVM achieved 83.69% and 83.57%. This comparison revealed RandomForest as the superior algorithm for the Studio 3 classification task, demonstrating significantly better performance than the other methods.

Step 1: Data Collection

For this step, I extracted the specific columns required based on my student ID (104330455), which ends with 5. According to the assignment requirements, I worked with:

```

columns_to_extract = ['Frame',
                      'Right Upper Leg x', 'Right Upper Leg y', 'Right Upper Leg z',
                      'Left Upper Leg x', 'Left Upper Leg y', 'Left Upper Leg z']

boning_extracted_df = boning_df[columns_to_extract].copy()
slicing_extracted_df = slicing_df[columns_to_extract].copy()

boning_extracted_df['class'] = 0
slicing_extracted_df['class'] = 1

combined_df = pd.concat([boning_extracted_df, slicing_extracted_df], ignore_index = True)

combined_df.to_csv('combined_data_step1.csv', index=False)

```

Figure 1. Data Extraction and Combination Code.

- Column Set 1: Right Upper Leg (x,y,z)
- Column Set 2: Left Upper Leg (x,y,z)

I loaded the two data files containing acceleration values for boning and slicing activities. From each file, I extracted the relevant columns including the frame column and combined them into a single dataset. I then added class labels to distinguish between the two activities (0 for boning, 1 for slicing).

The resulting dataset contained 8 columns:

```

df = pd.read_csv('combined_data_step1.csv')

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72060 entries, 0 to 72059
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Frame                  72060 non-null  int64
1   Right Upper Leg x      72060 non-null  float64
2   Right Upper Leg y      72060 non-null  float64
3   Right Upper Leg z      72060 non-null  float64
4   Left Upper Leg x       72060 non-null  float64
5   Left Upper Leg y       72060 non-null  float64
6   Left Upper Leg z       72060 non-null  float64
7   class                  72060 non-null  int64
dtypes: float64(6), int64(2)
memory usage: 4.4 MB

df.shape

(72060, 8)

```

Figure 2. Step1 Data Verification.

- Frame
- Right Upper Leg (x, y, z) - 3 columns
- Left Upper Leg (x, y, z) - 3 columns
- Class

The dataset includes 54,180 frames from the boning activity and 17,880 frames from the slicing activity, providing a comprehensive collection of movement data for analysis.

Step 2: Composite Columns

In Step 2, I created additional composite columns to extract more meaningful information from the raw acceleration data. For both the Right Upper Leg and Left Upper Leg, I calculated the following derived measurements:

```
df = df.rename(columns={
    'Frame': 'frame',
    'Right Upper Leg x': 'right_leg_x',
    'Right Upper Leg y': 'right_leg_y',
    'Right Upper Leg z': 'right_leg_z',
    'Left Upper Leg x': 'left_leg_x',
    'Left Upper Leg y': 'left_leg_y',
    'Left Upper Leg z': 'left_leg_z'
})

# 1. Root mean square value of x and y
df['rms_xy_right'] = np.sqrt((df['right_leg_x']**2 + df['right_leg_y']**2) / 2)
df['rms_xy_left'] = np.sqrt((df['left_leg_x']**2 + df['left_leg_y']**2) / 2)

# 2. Root mean square value of y and z
df['rms_yz_right'] = np.sqrt((df['right_leg_y']**2 + df['right_leg_z']**2) / 2)
df['rms_yz_left'] = np.sqrt((df['left_leg_y']**2 + df['left_leg_z']**2) / 2)

# 3. Root mean square value of z and x
df['rms_zx_right'] = np.sqrt((df['right_leg_z']**2 + df['right_leg_x']**2) / 2)
df['rms_zx_left'] = np.sqrt((df['left_leg_z']**2 + df['left_leg_x']**2) / 2)

# 4. Root mean square value of x y z
df['rms_xyz_right'] = np.sqrt((df['right_leg_x']**2 + df['right_leg_y']**2 + df['right_leg_z']**2) / 3)
df['rms_xyz_left'] = np.sqrt((df['left_leg_x']**2 + df['left_leg_y']**2 + df['left_leg_z']**2) / 3)

# 5. Roll calculation
df['roll_right'] = 180 * np.arctan2(df['right_leg_y'],
                                   np.sqrt(df['right_leg_x']**2 + df['right_leg_z']**2)) / np.pi
df['roll_left'] = 180 * np.arctan2(df['left_leg_y'],
                                   np.sqrt(df['left_leg_x']**2 + df['left_leg_z']**2)) / np.pi

# 6. Pitch calculation
df['pitch_right'] = 180 * np.arctan2(df['right_leg_x'],
                                   np.sqrt(df['right_leg_y']**2 + df['right_leg_z']**2)) / np.pi
df['pitch_left'] = 180 * np.arctan2(df['left_leg_x'],
                                   np.sqrt(df['left_leg_y']**2 + df['left_leg_z']**2)) / np.pi

df = df.to_csv('combined_data_step2.csv', index=False)
```

Figure 3. Composite Columns Code.

These calculations were performed separately for each leg to capture their distinct movement patterns. The RMS values provide a measure of the combined magnitude of acceleration across different axes, while roll and pitch values represent the angular orientation of the legs during the meat processing activities.

The enhanced dataset now contained 20 columns in total:

```
df = pd.read_csv('combined_data_step2.csv')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72060 entries, 0 to 72059
Data columns (total 20 columns):
#   Column              Non-Null Count  Dtype
---  -
0   frame               72060 non-null  int64
1   right_leg_x         72060 non-null  float64
2   right_leg_y         72060 non-null  float64
3   right_leg_z         72060 non-null  float64
4   left_leg_x          72060 non-null  float64
5   left_leg_y          72060 non-null  float64
6   left_leg_z          72060 non-null  float64
7   class               72060 non-null  int64
8   rms_xy_right        72060 non-null  float64
9   rms_xy_left         72060 non-null  float64
10  rms_yz_right        72060 non-null  float64
11  rms_yz_left         72060 non-null  float64
12  rms_zx_right        72060 non-null  float64
13  rms_zx_left         72060 non-null  float64
14  rms_xyz_right       72060 non-null  float64
15  rms_xyz_left        72060 non-null  float64
16  roll_right          72060 non-null  float64
17  roll_left           72060 non-null  float64
18  pitch_right         72060 non-null  float64
19  pitch_left          72060 non-null  float64
dtypes: float64(18), int64(2)
memory usage: 11.0 MB
```

```
df.shape
```

```
(72060, 20)
```

Figure 4. Step2 Data Verification.

- Frame (1 column)
- Original sensor data (6 columns)
- Composite features (12 columns)
- Class label (1 column)

This expanded feature set provides a richer representation of the movement patterns for subsequent analysis.

Step 3: Data Pre-processing

After creating the composite columns, I proceeded to compute statistical features per minute of data (60 frames = 1 minute) as specified in the assignment.

For each of the 18 columns (excluding frame and class), I calculated six statistical features:

```
def compute_features(df_segment, column_name):
    features = {}

    # Mean
    features[f'mean_{column_name}'] = df_segment[column_name].mean()

    # Standard deviation
    features[f'std_{column_name}'] = df_segment[column_name].std()

    # Min and Max
    features[f'min_{column_name}'] = df_segment[column_name].min()
    features[f'max_{column_name}'] = df_segment[column_name].max()

    # Area under the curve (AUC) - using trapezoidal rule
    features[f'auc_{column_name}'] = integrate.trapz(df_segment[column_name].values)

    # Number of peaks
    # Using prominence parameter to avoid detecting noise as peaks
    peaks, _ = find_peaks(df_segment[column_name].values, prominence=0.1)
    features[f'peaks_{column_name}'] = len(peaks)

    return features
```

Figure 5. Feature Computation Function Implementation

1. Mean - capturing the average acceleration or orientation
2. Standard deviation - measuring the variability in movement
3. Minimum value - representing the lower bounds of movement
4. Maximum value - representing the upper bounds of movement
5. Area under the curve (AUC) - measuring the total "energy" of movement
6. Number of peaks - counting distinct movement patterns

For each minute-long segment of data, I created a feature vector containing all these statistical measures. This transformed the time-series data into a set of statistical features that describe the characteristic movements associated with each activity.

```
# Process data in one-minute chunks (60 frames per minute)
for class_value in [0, 1]:
    class_df = df[df['class'] == class_value]

    for i in range(0, len(class_df), 60):
        minute_df = class_df.iloc[i:i+60]
        if len(minute_df) < 60:
            continue
```

Figure 6. Statistical Feature Computation Initialization.

The resulting feature dataset contained 108 features (18 columns $\times$ 6 statistical features) plus the class label (109 features), creating a rich representation for distinguishing between boning and slicing

activities. This preprocessing step reduced the data dimensionality while preserving the essential patterns that differentiate the two activities.

```
features_df = pd.read_csv('combined_data_step3.csv')
```

```
features_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1201 entries, 0 to 1200
Columns: 109 entries, class to peaks_pitch_left
dtypes: float64(90), int64(19)
memory usage: 1022.9 KB
```

```
features_df.shape
```

```
(1201, 109)
```

Figure 7. Step3 Data Verification.

Step 4: Training

Using the feature dataset with 108 features, I trained various classifiers following the approach from Studio 3. First, I trained SVM classifiers with five different configurations:

1. Basic SVM with 70/30 train-test split

1. SVM With Train-Test Split (70/30)

```
: svm = SVC(random_state=42)
   svm.fit(X_train, y_train)
   svm_tt = evaluate_model(svm, X_test, y_test, "SVM (Train-Test split)")
```

SVM (Train-Test split) Results:
Accuracy: 0.8975

Figure 8. SVM 70-30 Split Performance Metric.

This basic SVM model achieved good performance with an accuracy of 89.75%. It established a strong baseline for comparison with more complex configurations.

2. SVM with 10-fold cross-validation

2. SVM with 10-fold Cross-Validation

```
svm_cv = SVC(random_state=42)
cv_scores = cross_val_score(svm_cv, X_scaled, y, cv=10)
svm_cv = cv_scores.mean()
print(f"\nSVM with 10-fold Cross-Validation:")
print(f"CV Accuracy: {svm_cv:.4f}")
```

SVM with 10-fold Cross-Validation:
CV Accuracy: 0.8951

Figure 9. SVM 10-fold Cross-Validation Performance Metric.

The cross-validation approach produced a mean accuracy of 89.51%. This indicates consistent performance across different data splits, validating the robustness of the SVM model for this classification task.

3. SVM with hyperparameter tuning (optimizing C, gamma, and kernel)

3. SVM with Hyperparameter Tuning

```
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 'auto', 0.01, 0.001],
    'kernel': ['rbf', 'linear']
}
grid_search = GridSearchCV(SVC(random_state=42), param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

print("\nSVM Hyperparameter Tuning Results:")
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.4f}")

best_svm = grid_search.best_estimator_
svm_tuned_tt = evaluate_model(best_svm, X_test, y_test, "SVM Tuned (Train-Test Split)")
svm_tuned_cv = grid_search.best_score_
```

SVM Hyperparameter Tuning Results:
Best parameters: {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
Best cross-validation score: 0.9000

SVM Tuned (Train-Test Split) Results:
Accuracy: 0.9058

Figure 10. SVM Hyperparameter Tuning Performance Metric.

After grid search, the optimal parameters were found to be C=10, gamma='scale', and kernel='rbf'. The tuned model achieved the highest accuracy among SVM models at 90.58%, with a cross-validation score of 90.00%. This improvement suggests that hyperparameter tuning effectively optimized the model's performance for this dataset.

4. SVM with hyperparameter tuning and the 10 best features selected using ANOVA F-value

4. SVM with Feature Selection

```

selector = SelectKBest(f_classif, k=10)
X_train_selected = selector.fit_transform(X_train, y_train)
X_test_selected = selector.transform(X_test)

# Get selected feature names for reference
selected_indices = selector.get_support(indices=True)
selected_features = X.columns[selected_indices]
print("\nSelected Features:")
print(selected_features.tolist())

best_svm_fs = SVC(**grid_search.best_params_, random_state=42)
best_svm_fs.fit(X_train_selected, y_train)
svm_fs_tt = evaluate_model(best_svm_fs, X_test_selected, y_test, "SVM Tuned with Feature Selection")

Selected Features:
['peaks_right_leg_x', 'peaks_right_leg_z', 'peaks_left_leg_x', 'peaks_left_leg_y', 'peaks_left_leg_z', 'peaks_rms_xy_right', 'peaks_rms_xz_right', 'peaks_rms_zx_left',
'peaks_rms_xyz_right', 'peaks_pitch_right']

SVM Tuned with Feature Selection Results:
Accuracy: 0.8560

```

Figure 11. SWM Feature Selection Performance Metric.

Feature selection identified key features primarily related to peaks in both legs' movement patterns. This streamlined model achieved an accuracy of 85.60%. While the performance was slightly lower than the full-featured model, it demonstrates that a subset of critical features can still provide reasonable classification accuracy.

5. SVM with hyperparameter tuning and 10 principal components from PCA

5. SVM with PCA

```

pca = PCA(n_components=10)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

print(f"\nPCA Explained Variance: {sum(pca.explained_variance_ratio_):.4f}")

best_svm_pca = SVC(**grid_search.best_params_, random_state=42)
best_svm_pca.fit(X_train_pca, y_train)
svm_pca_tt = evaluate_model(best_svm_pca, X_test_pca, y_test, "SVM Tuned with PCA")

PCA Explained Variance: 0.7258

SVM Tuned with PCA Results:
Accuracy: 0.9003

```

Figure 12. SWM PCA Performance Metric.

The PCA approach captured 72.58% of the data variance using just 10 components. This model achieved an accuracy of 90.03%, nearly matching the performance of the best hyperparameter-tuned model while significantly reducing dimensionality.

I then trained three additional classifiers on the original dataset:

1. Stochastic Gradient Descent (SGD) classifier

SGD Classifier

```
: sgd = SGDClassifier(max_iter=1000, random_state=42)
sgd.fit(X_train, y_train)
sgd_tt = evaluate_model(sgd, X_test, y_test, "SGD Classifier")
```

SGD Classifier Results:
Accuracy: 0.8698

SGD with Cross-Validatiton

```
: sgd_cv = SGDClassifier(max_iter=1000, random_state=42)
sgd_cv_scores = cross_val_score(sgd_cv, X_scaled, y, cv=10)
sgd_cv = sgd_cv_scores.mean()
print(f"\nSGD Classifier with 10-fold Cross-Validation:")
print(f"CV Accuracy: {sgd_cv:.4f}")
```

SGD Classifier with 10-fold Cross-Validation:
CV Accuracy: 0.8402

Figure 13. SGD Classifier Implementation Results.

The SGD classifier achieved an accuracy of 86.98% with train-test split and 84.02% with cross-validation. While this model showed adequate performance, it was lower than most SVM configurations.

2. Random Forest classifier with 100 estimators

Random Forest

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_tt = evaluate_model(rf, X_test, y_test, "Random Forest")
```

Random Forest Results:
Accuracy: 0.8837

Random Forestt with Cross-Validation

```
rf_cv = RandomForestClassifier(n_estimators=100, random_state=42)
rf_cv_scores = cross_val_score(rf_cv, X_scaled, y, cv=10)
rf_cv = rf_cv_scores.mean()
print(f"\nRandom Forest with 10-fold Cross-Validation:")
print(f"CV Accuracy: {rf_cv:.4f}")
```

Random Forest with 10-fold Cross-Validation:
CV Accuracy: 0.8718

Figure 14. Random Forest Implementation Results.

The Random Forest model delivered an accuracy of 88.37% with train-test split and 87.18% with cross-validation. Although this model didn't achieve the highest overall accuracy, its ensemble approach effectively captured important patterns in the movement data.

3. Multi-layer Perceptron (MLP) neural network with two hidden layers (100, 50)

MLP Neural Network

```
: mlp = MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=1000, random_state=42)
  mlp.fit(X_train, y_train)
  mlp_tt = evaluate_model(mlp, X_test, y_test, "MLP Neural Network")
```

MLP Neural Network Results:
Accuracy: 0.8809

MLP Neural Network with Cross-Validation

```
: mlp_cv = MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=1000, random_state=42)
  mlp_cv_scores = cross_val_score(mlp_cv, X_scaled, y, cv=10)
  mlp_cv = mlp_cv_scores.mean()
  print(f"\nMLP Neural Network with 10-fold Cross-Validation:")
  print(f"CV Accuracy: {mlp_cv:.4f}")
```

MLP Neural Network with 10-fold Cross-Validation:
CV Accuracy: 0.8842

Figure 15. MLP Neural Network Implementation Results.

The MLP neural network achieved an accuracy of 88.09% with train-test split and 88.42% with cross-validation. This model showed strong and consistent performance across both evaluation methods, demonstrating the neural network's ability to learn complex patterns in high-dimensional sensor data.

All models were evaluated using accuracy as the primary metric, with results summarized in the tables below:

SVM Models Summary Table

	SVM model	Train-test split	Cross-validation
0	Train-Test split (70/30)	89.75%	N/A
1	10-fold cross-validation	N/A	89.51%
2	1 and 2 with hyperparameter tuning	90.58%	90.00%
3	1 and 2 with hyperparameter tuning and 10 bes...	85.60%	90.00%
4	1 and 2 with hyperparameter tuning and 10 pri...	90.03%	90.00%

Figure 16. SVM Summary Table.

Studio Activity 7: Other Classifiers Summary Table

	Model	Train-test split	Cross-validation
0	SVM	89.75%	89.51%
1	SGD	86.98%	84.02%
2	RandomForest	88.37%	87.18%
3	MLP	88.09%	88.42%

Figure 17. Models Summary Table.

Step 5: Model Selection

Based on the training results, I analyzed the performance of the various models to determine the best approach for classifying boning and slicing activities.

- For the SVM models, the hyperparameter-tuned SVM with train-test split achieved the highest accuracy of 90.58%. This represents a significant improvement over the basic SVM model, demonstrating the importance of parameter optimization for this classification task. The hyperparameter tuning process identified that an RBF kernel with $C=10$ and $\gamma='scale'$ provided the optimal decision boundary for separating the two activity classes.

```
Best SVM Model:
SVM model          1 and 2 with hyperparameter tuning
Train-test split   90.58%
Cross-validation    90.00%
Name: 2, dtype: object
```

Figure 17. Best SVM Model.

- When comparing across all model types, the hyperparameter-tuned SVM remained the best performer. While the Random Forest (88.37%) and MLP (88.09%) models showed competitive performance, they didn't surpass the optimized SVM. The SGD classifier performed the poorest among all models with 86.98% accuracy.

```
Best Overall Model:
Model                SVM
Train-test split    89.75%
Cross-validation    89.51%
Name: 0, dtype: object
```

Figure 18. Best Overall Model.

Based on these results, I would select the hyperparameter-tuned SVM model (90.58% accuracy) for deployment due to its superior classification performance. For applications where computational efficiency is a concern, the PCA-based SVM model (90.03% accuracy) would be an excellent alternative, as it achieves nearly identical performance while working with a significantly reduced feature set.

Appendix: Source Code Repository

The complete source code for this project is available at the following link:

https://github.com/ThuanDancoi/COS40007/blob/main/Portfolio%20Assessment%202/code/portfolio_assessment2.ipynb

References

<https://www.kaggle.com/code/faressayah/support-vector-machine-pca-tutorial-for-beginner>

<https://www.geeksforgeeks.org/svm-hyperparameter-tuning-using-gridsearchcv-ml/>

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

<https://help.alteryx.com/current/en/designer/tools/predictive/model-comparison-tool.html#idp364826>